

Gerenciando dados textuais com pandas

Trabalhando com Texto em Python I · Unidade 5

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

2026

Sumário

1. Importando dados para o pandas	1
2. Examinando DataFrames	4
3. Estendendo DataFrames	6
4. Salvando DataFrames	9
Quiz	10
Resumo da unidade	11

Objetivos de aprendizagem

Ao final desta unidade, você deverá saber:

- **importar dados** para um `DataFrame` do pandas;
- **explorar** dados armazenados em um `DataFrame`;
- **acrescentar** (estender) dados a um `DataFrame`;
- **salvar** os dados de um `DataFrame`.

O **pandas** é uma biblioteca do Python para trabalhar com **dados tabulares**. Começamos importando-a — usamos `as` para controlar o nome do módulo importado; por convenção, o pandas é abreviado como `pd`:

```
import pandas as pd
```

1. Importando dados para o pandas

1.1 A partir de um único arquivo

Formatos típicos para distribuir corpora incluem arquivos **CSV** (*Comma-separated Values*, valores separados por vírgula), **JSON** (*JavaScript Object Notation*) e texto simples. O pandas oferece muitas funções para ler dados em vários formatos — dá até para importar planilhas do Excel!

O exemplo a seguir carrega um recorte do *SFU Opinion and Comments Corpus* (SOCC), que reúne artigos de opinião do jornal canadense *The Globe and Mail*. Usamos a função `read_csv()`, que recebe uma *string* com o **caminho** do arquivo:

```
# Lê o arquivo CSV e atribui a saída a variável 'socc'
socc = pd.read_csv('data/socc_gnm_articles.csv')
```

```
# Examina o tipo do objeto guardado em 'socc'
type(socc)
```

```
pandas.core.frame.DataFrame
```

O pandas faz todo o trabalho pesado e devolve o conteúdo do CSV em um **DataFrame**, a estrutura de dados nativa do pandas. Usamos o método `head()` para ver as primeiras cinco linhas:

```
# Imprime as primeiras cinco linhas do DataFrame
socc.head(5)
```

O **DataFrame** tem forma **tabular**, com colunas como `article_id`, `title`, `author`, `article_text` etc., além de um **índice** para cada linha (0, 1, 2, ...). Uma versão compacta (algumas colunas omitidas para caber):

índice	article_id	title	author	ntop_level_comments
0	26842506	The Tories deserve another mandate...	GLOBE EDITORIAL	1378.0
1	26055892	Harper hysteria a sign of closed liberal minds	Konrad Yakabuski	455.0
2	6929035	Too many first nations people live in a dream...	Jeffrey Simpson	433.0
3	19047636	The Globe's editorial board endorses Tim Hudak...	GLOBE EDITORIAL	432.0
4	11672346	Disgruntled Arab states look to strip Canada...	Campbell Clark	411.0

O acessador `.at[]` permite inspecionar **um único item**. Vamos ver o valor da coluna `title` no índice 123:

```
socc.at[123, 'title']
```

```
"How Toronto got a 'world-class,' gold-plated, half-billion-dollar empty train"
```

1.2 A partir de múltiplos arquivos

Outro cenário comum é ter **vários arquivos** de texto para carregar. Primeiro, coletamos os arquivos com a classe **Path** (vista na Unidade 1):

```
# Importa a classe Path
from pathlib import Path

# Cria um objeto Path que aponta para o diretório com os dados
corpus_dir = Path('data')
```

```
# Coleta todos os arquivos .txt no diretório do corpus
corpus_files = list(corpus_dir.glob('*.txt'))

# Verifica os arquivos do corpus
corpus_files
```

```
[PosixPath('data/WP_1990-08-10-25A.txt'),
 PosixPath('data/NYT_1991-01-16-A15.txt'),
 PosixPath('data/WP_1991-01-17-A1B.txt')]
```

Para acomodar os dados, criamos um **DataFrame** **vazio** e definimos sua forma de antemão: o número de linhas (**index**) e os nomes das colunas (**columns**). Determinamos o número de linhas com a função **range()**, entre 0 e a quantidade de arquivos (obtida com **len()**). Para as colunas, passamos uma lista de *strings* ao argumento **columns**:

```
# Cria um DataFrame e atribui o resultado a variável 'df'
df = pd.DataFrame(index=range(0, len(corpus_files)), columns=['filename', 'text'])

# Chama a variável para inspecionar a saída
df
```

índice	filename	text
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN

Agora percorremos os objetos **Path** em **corpus_files**, lemos o conteúdo e o adicionamos ao **DataFrame** com o acessador **.at**:

```
# Percorre os arquivos do corpus e conta cada volta com enumerate()
for i, f in enumerate(corpus_files):

    # Lê o conteúdo do arquivo
    text = f.read_text(encoding='utf-8')

    # Pega o nome do arquivo do objeto Path
    filename = f.name

    # Atribui o texto do arquivo ao índice 'i' na coluna 'text' usando
    # o acessador .at -- isso modifica o DataFrame "no lugar" (in place);
    # não é preciso atribuir o resultado a uma variável
    df.at[i, 'text'] = text

    # Faz o mesmo com o nome do arquivo
    df.at[i, 'filename'] = filename
```

O **DataFrame** fica populado com os nomes dos arquivos e o texto:

índice	filename	text
0	WP_1990-08-10-25A.txt	*We Don't Stand for Bullies': Diverse Voices...
1	NYT_1991-01-16-A15.txt	U.S. TAKING STEPS TO CURB TERRORISM: F.B.I. I...
2	WP_1991-01-17-A1B.txt	U.S., Allies Launch Massive Air War Against T...

2. Examinando DataFrames

Os `DataFrames` guardam muita informação, geralmente organizada em **colunas**, acessíveis pelo atributo `columns`:

```
# Recupera as colunas e seus nomes
socc.columns
```

```
Index(['article_id', 'title', 'article_url', 'author', 'published_date',
      'ncomments', 'ntop_level_comments', 'article_text'],
      dtype='object')
```

Uma coluna inteira é acessada com colchetes `[]`, colocando o nome da coluna como *string* — como as chaves de um dicionário. Vamos recuperar a coluna `author`:

```
# Recupera o conteúdo da coluna 'author' no DataFrame 'socc'
socc['author']
```

```
0      GLOBE EDITORIAL
1      Konrad Yakabuski
2      Jeffrey Simpson
...
10337    Adam Radwanski
10338    GLOBE EDITORIAL
Name: author, Length: 10339, dtype: object
```

A coluna contém 10 339 objetos (veja `Length` e `dtype`); os números à esquerda são o índice. Cada coluna de um `DataFrame` é, na verdade, um objeto `Series` — pense no `DataFrame` como a tabela inteira, cujas colunas são `Series`:

```
# Verifica o tipo de 'socc' e de 'socc['author']'
type(socc), type(socc['author'])
```

```
(pandas.core.frame.DataFrame, pandas.core.series.Series)
```

Nota: ao imprimir um `DataFrame` ou `Series`, o pandas **omite** tudo entre as cinco primeiras e as cinco últimas linhas por padrão — conveniente ao trabalhar com milhares de linhas.

O método `value_counts()` conta os valores únicos de uma `Series`:

uma coluna = Series

índice	filename	ncomments	author
0	WP_1990.txt	2187.0	GLOBE ED.
1	NYT_1991.txt	1103.0	K. Yakabuski
2	WP_1991.txt	1164.0	J. Simpson

linha
(um registro)

A tabela inteira é um **DataFrame**; cada coluna é uma **Series**.

Figura 1: Anatomia de um **DataFrame**: a tabela inteira é um **DataFrame**, cada **coluna** é uma **Series**, cada **linha** é um registro e a coluna cinza à esquerda é o **índice**.

```
# Conta os valores únicos na coluna 'author'
socc['author'].value_counts()
```

```
GLOBE EDITORIAL          2712
Jeffrey Simpson          649
Margaret Wente           547
...
Jessica Scott-Reid        1
Kenneth Oppel             1
Name: author, Length: 1896, dtype: int64
```

Sem surpresa, a equipe editorial do *The Globe and Mail* assina a maioria dos editoriais! Podemos **visualizar** o resultado chamando o método `.plot()`, que usa a biblioteca **matplotlib**. Passamos a *string* `bar` ao argumento `kind` para um gráfico de barras, e `[:10]` para os dez autores mais prolíficos:

```
# Magia do Jupyter que permite renderizar graficos do matplotlib no notebook!
# Voce so precisa executar este comando uma vez.
%matplotlib inline

# Conta os valores da coluna 'author' e limita ao top-10 antes de plotar.
socc['author'].value_counts()[:10].plot(kind='bar')
```

Dica: o `.plot()` gera uma **imagem** (gráfico de barras) exibida no notebook; a saída de texto é apenas `<AxesSubplot: >`. Para outros tipos de gráfico, mude o argumento `kind` (ex.: `'line'`, `'hist'`, `'pie'`).

Para colunas **numéricas**, o método `describe()` dá estatísticas descritivas básicas:

```
# Estatísticas descritivas básicas da coluna 'ntop_level_comments'
socc['ntop_level_comments'].describe()
```

```
count    10339.000000
mean      26.384273
std       39.786923
min        0.000000
25%        1.000000
50%       14.000000
75%       35.000000
max      1378.000000
Name: ntop_level_comments, dtype: float64
```

Lendo a saída: há 10 339 linhas; a média (**mean**) de comentários por editorial é ≈ 26 , mas com forte variação (desvio-padrão **std** de quase 40). O mínimo (**min**) é 0 (alguns editoriais não têm comentários). O primeiro quartil (**25%**) mostra que 25% dos dados têm 1 comentário ou menos; a mediana (**50%**) é 14; o terceiro quartil (**75%**) é 35; e o mais comentado (**max**) tem 1 378 comentários.

2.1 Filtrando linhas com `.loc`

Como encontrar os artigos com **zero** comentários? Usamos o acessador `.loc` para selecionar linhas com base em seus valores. Como `=` é reservado para atribuição, a comparação “é igual a” usa **dois** sinais (`==`):

```
# Pega as linhas sem comentários de nível superior
socc.loc[socc['ntop_level_comments'] == 0]
```

```
[... 2542 rows x 8 columns ...]
```

Isso retorna 2 542 linhas em que `ntop_level_comments` é zero. Para visões mais complexas, combinamos critérios com o operador `&` (“E” lógico) — cada critério deve ficar entre **parênteses** `()`. Vamos verificar se o primeiro autor do resultado (Hayden King) escreveu outros artigos com zero comentários:

```
# Número de comentários de nível superior para o autor Hayden King
socc.loc[(socc['ntop_level_comments'] == 0) & (socc['author'] == 'Hayden King')]
```

Isso retorna apenas a linha de índice 7797 — o único artigo dele sem comentários.

3. Estendendo DataFrames

É fácil **acrescentar** informação a um `DataFrame`. Um cenário comum: carregar dados de um arquivo, fazer análises e guardar os resultados no mesmo `DataFrame`. Adicionamos uma coluna vazia com o acessador de coluna `[]` e o tipo `None`:

```
# Adiciona uma nova coluna chamada 'comments_ratio' ao DataFrame
socc['comments_ratio'] = None
```

Vamos **preencher** a coluna calculando a proporção de comentários de nível superior (comentários sobre o artigo) em relação a todos os comentários — basta dividir uma coluna pela outra:

```
# Preenche a coluna 'comments_ratio' com a razão entre comentários de
# nível superior e o total de comentários
```

```
socc['comments_ratio'] = socc['ntop_level_comments'] / socc['ncomments']
```

Os acessadores de coluna podem ser usados de forma muito flexível para acessar e manipular dados, como nesta divisão. Uma amostra do resultado:

índice	ntop_level_comments	ncomments	comments_ratio
0	1378.0	2187.0	0.630087
1	455.0	1103.0	0.412511
2	433.0	1164.0	0.371993

Mas alguns artigos **não** receberam comentários — nesses casos, teríamos dividido zero por zero:

```
# Imprime os cinco primeiros artigos sem comentários de nível superior
socc.loc[socc['ntop_level_comments'] == 0].head(5)
```

Para essas linhas, `comments_ratio` fica marcada como **NaN** (*not a number*, “não é um número”): a divisão foi feita, mas o resultado não é um número. O pandas **ignora automaticamente** os **NaN** nos cálculos, como mostra o `describe()`:

```
# Estatísticas descritivas da coluna 'comments_ratio'
socc['comments_ratio'].describe()
```

```
count      7797.000000
mean        0.537057
std         0.205398
min         0.083333
25%         0.384615
50%         0.485714
75%         0.647059
max         1.000000
Name: comments_ratio, dtype: float64
```

Note a diferença no `count`: só 7 797 itens (dos 10 339) entraram no cálculo — os **NaN** foram ignorados.

3.1 Aplicando processamento de linguagem a uma coluna

E se quiséssemos fazer PLN e guardar os resultados no `DataFrame`? Vamos selecionar artigos no primeiro quartil de `comments_ratio` e com mais de 200 comentários (`ncomments`), usando `&`:

```
# Filtra o DataFrame por artigos muito comentados e atribui o resultado a 'talk'
talk = socc.loc[(socc['comments_ratio'] <= 0.384) & (socc['ncomments'] >= 200)]
```

Importamos o `spaCy` e carregamos um modelo **médio** para o inglês:

```
# Importa a biblioteca spaCy
import spacy

# Note que agora carregamos um modelo de tamanho medio!
nlp = spacy.load('en_core_web_md')
```

Ao criar uma coluna nova em `talk`, o pandas emite um `SettingWithCopyWarning`, porque `talk` é apenas um recorte (*slice*) do `DataFrame` original. Atribuir uma coluna a apenas uma parte quebraria a estrutura

tabular. A solução é criar uma **cópia profunda** (*deep copy*) do recorte com o método `.copy()`:

```
# Cria uma copia profunda do DataFrame
talk = talk.copy()

# Cria uma nova coluna chamada 'processed_title'
talk['processed_title'] = None
```

Para processar os títulos, usamos o método `apply()` de um `DataFrame`, que **aplica** o que recebe a cada linha da coluna. Passamos o modelo `nlp` — ou seja, aplicamos o modelo aos títulos (que são *strings*) da coluna `title`:

```
# Aplica o modelo de linguagem 'nlp' ao conteúdo da coluna 'title'
talk['processed_title'] = talk['title'].apply(nlp)
```

Cada célula da coluna `processed_title` agora contém um objeto `Doc` do `spaCy`:

```
# Pega o valor da coluna 'processed_title' na linha de índice 2
talk.at[2, 'processed_title']

# Verifica o tipo do objeto contido
type(talk.at[2, 'processed_title'])
```

```
Too many first nations people live in a dream palace
spacy.tokens.doc.Doc
```

3.2 Definindo a própria função e aplicando com `apply()`

Vamos definir nossa própria função para buscar os **lemas de cada substantivo** do título. Funções em Python são definidas com `def`, seguido do nome e dos parâmetros entre parênteses:

```
# Define uma função 'get_nouns' que recebe um único objeto como entrada.
# Referimo-nos a essa entrada pela variável 'nlp_text'.
def get_nouns(nlp_text):

    # Primeiro garantimos que a entrada é do tipo correto,
    # usando 'assert' para checar o tipo
    assert type(nlp_text) == spacy.tokens.doc.Doc

    # Prepara uma lista vazia para os lemas
    lemmas = []

    # Percorre o objeto Doc
    for token in nlp_text:

        # Se a classe gramatical detalhada do token for substantivo (NN)
        if token.tag_ == 'NN':

            # Acrescenta o lema do token a lista de lemas
            lemmas.append(token.lemma_)

    # Ao fim do laço, retorna a lista de lemas
    return lemmas
```

Aplicamos a função com `apply()`, criando a coluna `nouns` automaticamente pela atribuição:

```
# Aplica a função 'get_nouns' a coluna 'processed_title'
talk['nouns'] = talk['processed_title'].apply(get_nouns)
```

Uma amostra dos títulos processados e seus substantivos:

índice	title	nouns
2	Too many first nations people live in a dream...	[dream, palace]
5	Fifty years in Canada, and now I feel like a s...	[class, citizen]
8	A nation of \$100,000 firefighters	[nation]

Nota: não é necessário criar uma coluna vazia antes — o pandas cria a coluna nova automaticamente na atribuição, como em `talk['nouns']`.

3.3 Extraíndo dados para estruturas nativas do Python

O método `tolist()` extrai o conteúdo de uma `Series` para uma **lista**:

```
# Converte a pandas Series em uma lista
noun_list = talk['nouns'].tolist()
```

```
# Mostra os dez primeiros
noun_list[:10]
```

```
[['dream', 'palace'], ['agency'], ['class', 'citizen'], ['right'],
 ['nation'], [], ['reform'], ['leader', 'parade'], ['pm'],
 ['government', 'monopoly']]
```

Temos uma **lista de listas** (cada linha da coluna `nouns` é uma lista). Vamos “achatar” tudo em uma única lista `final_list` com o método `extend()`:

```
# Prepara a lista vazia
final_list = []

# Percorre cada lista dentro da lista de listas
for nlist in noun_list:

    # Estende a lista final com a lista atual
    final_list.extend(nlist)

# Verifica o tamanho da lista
len(final_list)
```

884

Para plotar os 10 substantivos mais frequentes, convertamos `final_list` em uma `Series`, contamos com `value_counts()` e plotamos:

```
# Converte a lista em uma pandas Series, conta os substantivos únicos com
# value_counts(), pega os 10 mais frequentes [:10] e plota em barras.
pd.Series(final_list).value_counts()[:10].plot(kind='bar')
```

4. Salvando DataFrames

DataFrames podem ser salvos como objetos **serializados** (*pickled*) com o método `to_pickle()`, que recebe o caminho do arquivo. Vamos salvar o `df` com os três artigos:

```
# Grava o DataFrame em disco usando pickle
df.to_pickle('data/pickled_df.pkl')
```

Segurança (importante no CiberExt): o formato *pickle* serializa objetos Python arbitrários e, ao ser **carregado**, pode **executar código arbitrário**. **Nunca** faça `read_pickle()` (nem `pickle.load`, `joblib.load`, `numpy` com `allow_pickle=True` etc.) em arquivos de **fontes não confiáveis** — um `.pkl` malicioso compromete a máquina. Use *pickle* apenas com dados que **você mesmo** gerou; para intercâmbio entre pessoas/sistemas, prefira formatos que não executam código, como **CSV**, **JSON** ou **Parquet** (`to_csv/read_csv`, `to_json/read_json`, `to_parquet/read_parquet`).

Verificamos lendo de volta com `read_pickle()`:

```
# Lê o DataFrame serializado e atribui o resultado a 'df_2'
df_2 = pd.read_pickle('data/pickled_df.pkl')
```

Comparando os dois **DataFrames** com `==`, obtemos um valor booleano (**True/False**) para cada célula:

```
# Compara os DataFrames 'df' e 'df_2'
df == df_2
```

índice	filename	text
0	True	True
1	True	True
2	True	True

Tudo **True** — os dados foram salvos e recarregados com sucesso.

Quiz

Marque a alternativa correta (a resposta certa está destacada com ✓).

1. Por convenção, o **pandas** é importado com qual apelido?

1. `pd` ✓
2. `pandas`
3. `dt`

2. Cada coluna de um **DataFrame** é, na verdade, um objeto:

1. `Series` ✓
2. `DataFrame`
3. `list`

3. Um valor `NaN` indica:

1. Um valor ausente (“não é um número”) ✓
2. O número zero
3. Infinito

4. Qual operador combina dois critérios num filtro `.loc`?

1. `&` ✓
2. `and`
3. `+`

5. Por que criar uma cópia com `.copy()` de um recorte do `DataFrame`?

1. Para evitar o `SettingWithCopyWarning` e não alterar o original ✓
2. Para acelerar o cálculo
3. É obrigatório em toda operação

6. (Segurança) Por que *não* usar `read_pickle` em um arquivo de origem não confiável?

1. Porque pode executar código arbitrário na sua máquina ✓
2. Porque é mais lento que o CSV
3. Porque não consegue ler texto

Resumo da unidade

Nesta unidade, você aprendeu a:

1. **importar** dados para um `DataFrame` a partir de um único arquivo (`read_csv()`) ou de vários arquivos (`Path/glob` + laço com `.at`);
2. **examinar** `DataFrames`: `columns`, colunas como `Series`, `value_counts()`, `describe()`, `.plot()` e filtragem com `.loc` e critérios combinados (`&`, `==`);
3. **estender** `DataFrames`: adicionar colunas, calcular entre colunas, lidar com `NaN`, usar `.copy()` para evitar o `SettingWithCopyWarning`, e aplicar funções (do spaCy ou próprias) com `apply()`;
4. **extrair** dados com `tolist()/extend()` e **salvar/carregar** com `to_pickle()/read_pickle()`.

Exercícios sugeridos

1. Carregue os três arquivos de notícias da Unidade 1 em um `DataFrame` (colunas `filename` e `text`) e conte quantos caracteres cada texto tem, guardando o resultado em uma nova coluna `n_chars`.
2. No `DataFrame` `socc`, use `.loc` para selecionar os artigos do autor `GLOBE EDITORIAL` com mais de 500 comentários.
3. Escreva uma função `get_verbs()` (análoga a `get_nouns`) que colete os lemas dos **verbos** (`token.pos_ == 'VERB'`) e aplique-a a uma coluna de títulos processados.
4. Salve um `DataFrame` em CSV com `to_csv('saida.csv', index=False)` e recarregue-o com `read_csv()`, conferindo se as colunas foram preservadas.

Referências

- Rognvaldsson, E. et al. *Applied Language Technology* (MOOC). Universidade de Helsinque. <https://applied-language-technology.mooc.fi/>
- Kolhatkar, V. et al. (2020). The SFU Opinion and Comments Corpus. *Corpus Pragmatics*.
- McKinney, W. et al. *pandas: powerful Python data analysis toolkit*. <https://pandas.pydata.org/>